PCET Education Trust

Pimpri Chinchwad College OF Engineering, Pune

Database Management Systems Lab

ASSIGNMENT NO. 04

Name	PRN
Atharva Lokhande	124B1F009

Batch A1

Guided By: Mrs. Alpana Borse Ma'am

Case Study: Airline Management System

SQL Queries:

Part A: Procedures

A stored procedure is a precompiled set of SQL statements stored in the database that performs one or more operations such as inserting, updating, deleting, or retrieving data; it can accept input parameters, execute complex logic, and may or may not return values.

1. Add a New Booking

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddBooking (
    ->     IN p_passenger_id INT,
    ->     IN p_booking_date DATETIME
    -> )
    -> BEGIN
    ->     INSERT INTO Bookings (passenger_id, booking_date)
    ->     VALUES (p_passenger_id, p_booking_date);
    -> END //
ERROR 1304 (42000): PROCEDURE AddBooking already exists
mysql> DELIMITER ;
```

Call It:

```
mysql> CALL AddBooking(1, NOW());
Query OK, 1 row affected (0.05 sec)
```

2. Get All Flights for a Passenger

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetPassengerFlights (
    ->     IN p_passenger_id INT
    -> )
    -> BEGIN
    ->     SELECT f.flight_number, f.departure_time, f.arrival_time
    ->     FROM Flights f
    ->     JOIN Tickets t ON f.flight_id = t.flight_id
    ->     JOIN Bookings b ON t.booking_id = b.booking_id
    ->     WHERE b.passenger_id = p_passenger_id;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
```

Call It:

```
mysql> CALL GetPassengerFlights(1);
+---------------+---------------------+---------------------+
| flight_number | departure_time      | arrival_time        |
+---------------+---------------------+---------------------+
| AI101         | 2026-04-15 08:00:00 | 2026-04-15 10:00:00 |
+---------------+---------------------+---------------------+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.02 sec)
```

3. Update Passenger Email

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE UpdatePassengerEmail (
    ->     IN p_passenger_id INT,
    ->     IN p_new_email VARCHAR(100)
    -> )
    -> BEGIN
    ->     UPDATE Passengers
    ->     SET email = p_new_email
    ->     WHERE passenger_id = p_passenger_id;
    -> END //
Query OK, 0 rows affected (0.05 sec)

mysql> DELIMITER ;
```

Call It:

```
mysql> CALL UpdatePassengerEmail(1, 'newemail@example.com');
Query OK, 1 row affected (0.03 sec)
```

Part B: Functions

A function is a database object that takes input parameters, performs a calculation or computation, and always returns a single value; it is mainly used within SQL queries (like in SELECT statements) and does not modify database data.

1. Count Total Bookings of a Passenger

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetBookingCount (p_passenger_id INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total
    ->     FROM Bookings
    ->     WHERE passenger_id = p_passenger_id;
    ->
    ->     RETURN total;
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT GetBookingCount(1) AS total_bookings;
+----------------+
| total_bookings |
+----------------+
|              2 |
+----------------+
1 row in set (0.02 sec)
```

2. Get Aircraft Capacity by Flight

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION GetAircraftCapacity (p_flight_id INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE cap INT;
    ->
    ->     SELECT a.capacity INTO cap
    ->     FROM Aircraft a
    ->     JOIN Flights f ON a.aircraft_id = f.aircraft_id
    ->     WHERE f.flight_id = p_flight_id;
    ->
    ->     RETURN cap;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT GetAircraftCapacity(2);
+------------------------+
| GetAircraftCapacity(2) |
+------------------------+
|                    150 |
+------------------------+
1 row in set (0.01 sec)
```

3. Get Total Tickets in a Booking

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION GetTicketCount (p_booking_id INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->
    ->     SELECT COUNT(*) INTO total
    ->     FROM Tickets
    ->     WHERE booking_id = p_booking_id;
    ->
    ->     RETURN total;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT GetTicketCount(1) AS total_tickets;
+---------------+
| total_tickets |
+---------------+
|             1 |
+---------------+
1 row in set (0.00 sec)
```

Part C: Trigger

A **Trigger** is a database object that automatically executes **when a specific event happens** (INSERT, UPDATE, DELETE) on a table.

1. BEFORE INSERT Trigger

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_booking_insert
    -> BEFORE INSERT ON Bookings
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.passenger_id IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Passenger ID cannot be NULL';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> DELIMITER ;
```

2. AFTER INSERT Trigger

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_ticket_insert
    -> AFTER INSERT ON Tickets
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO Passengers (name, email, phone)
    ->     VALUES ('Notification', 'system@airline.com', '0000000000');
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
```

3. BEFORE DELETE Trigger

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER before_flight_delete
    -> BEFORE DELETE ON Flights
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE ticket_count INT;
    ->
    ->     SELECT COUNT(*) INTO ticket_count
    ->     FROM Tickets
    ->     WHERE flight_id = OLD.flight_id;
    ->
    ->     IF ticket_count > 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Cannot delete flight with existing tickets';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
```

4. AFTER UPDATE Trigger

First create a log table:

```
mysql> CREATE TABLE Passenger_Log (
    ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     passenger_id INT,
    ->     old_email VARCHAR(100),
    ->     new_email VARCHAR(100),
    ->     change_time DATETIME
    -> );
Query OK, 0 rows affected (0.11 sec)
```

Trigger:

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_passenger_update
    -> AFTER UPDATE ON Passengers
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.email <> NEW.email THEN
    ->         INSERT INTO Passenger_Log
    ->         VALUES (NULL, OLD.passenger_id, OLD.email, NEW.email, NOW());
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
```

Mini Project : Civic Voice Interface System

SQL Queries:

Part A : Procedures

1. Add New User (Secure Insert)

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddUser(
    ->     IN p_name VARCHAR(100),
    ->     IN p_email VARCHAR(100),
    ->     IN p_phone VARCHAR(15),
    ->     IN p_password VARCHAR(255),
    ->     IN p_role VARCHAR(20)
    -> )
    -> BEGIN
    ->     INSERT INTO User(full_name, email, phone, password_hash, role)
    ->     VALUES(p_name, p_email, p_phone, p_password, p_role);
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> CALL AddUser(
    ->     'Rohit Sharma',
    ->     'rohit.sharma99@gmail.com',
    ->     '8080104570',
    ->     'hashed_pass_456',
    ->     'citizen'
    -> );
Query OK, 1 row affected (0.01 sec)
```

2. Add Citizen Profile

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddCitizenProfile(
    ->     IN p_user_id INT,
    ->     IN p_address TEXT,
    ->     IN p_dob DATE,
    ->     IN p_gender VARCHAR(10),
    ->     IN p_aadhaar VARCHAR(20)
    -> )
    -> BEGIN
    ->     INSERT INTO CitizenProfile(user_id, address, date_of_birth, gender, aadhaar_number)
    ->     VALUES(p_user_id, p_address, p_dob, p_gender, p_aadhaar);
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> CALL AddCitizenProfile(
    ->     4,
    ->     'Pune, Maharashtra',
    ->     '2003-05-15',
    ->     'male',
    ->     '123456789012'
    -> );
```

3. Verify User Document

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE VerifyDocument(
    ->     IN p_document_id INT,
    ->     IN p_status VARCHAR(20)
    -> )
    -> BEGIN
    ->     UPDATE UserDocument
    ->     SET status = p_status
    ->     WHERE document_id = p_document_id;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> CALL VerifyDocument(
    ->     1,
    ->     'verified'
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Part B : Functions

1. Count Documents of a User

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetDocumentCount(p_user_id INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE doc_count INT;
    ->
    ->     SELECT COUNT(*) INTO doc_count
    ->     FROM UserDocument
    ->     WHERE user_id = p_user_id;
    ->
    ->     RETURN doc_count;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT GetDocumentCount(1) AS total_documents;
+-----------------+
| total_documents |
+-----------------+
|               2 |
+-----------------+
1 row in set (0.01 sec)
```

2. Check If User is Verified

```
mysql> DELIMITER //
mysql> CREATE FUNCTION IsUserVerified(p_user_id INT)
    -> RETURNS BOOLEAN
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE verified_count INT;
    ->
    ->     SELECT COUNT(*) INTO verified_count
    ->     FROM UserDocument
    ->     WHERE user_id = p_user_id AND status = 'verified';
    ->
    ->     IF verified_count > 0 THEN
    ->         RETURN TRUE;
    ->     ELSE
    ->         RETURN FALSE;
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT IsUserVerified(1) AS verification_status;
+---------------------+
| verification_status |
+---------------------+
|                   1 |
+---------------------+
1 row in set (0.01 sec)
```

3. Get User Age

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetUserAge(p_user_id INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE age INT;
    ->
    ->     SELECT TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())
    ->     INTO age
    ->     FROM CitizenProfile
    ->     WHERE user_id = p_user_id;
    ->
    ->     RETURN age;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

Use It:

```
mysql> SELECT GetUserAge(1) AS user_age;
+----------+
| user_age |
+----------+
|       22 |
+----------+
1 row in set (0.01 sec)
```

Part C : Trigger

1. Before Insert

```
mysql> DELIMITER //
mysql> CREATE TRIGGER BeforeUserInsert
    -> BEFORE INSERT ON User
    -> FOR EACH ROW
    -> BEGIN
    ->     IF EXISTS (
    ->         SELECT 1 FROM User WHERE email = NEW.email
    ->     ) THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Email already exists!';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

2. After Insert

Create New table UserLog

```
mysql> CREATE TABLE UserLog (
    ->     log_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     user_id INT,
    ->     action VARCHAR(100),
    ->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
```

Trigger

```
mysql> DELIMITER //
mysql> CREATE TRIGGER AfterUserInsert
    -> AFTER INSERT ON User
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO UserLog(user_id, action)
    ->     VALUES (NEW.user_id, 'New user created');
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql> DELIMITER ;
```

3. Before Delete

```
mysql> DELIMITER //
mysql> CREATE TRIGGER BeforeUserDelete
    -> BEFORE DELETE ON User
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.role = 'admin' THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Admin users cannot be deleted!';
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```

4. After Update

Create a New Table DocumentLog

```
mysql> CREATE TABLE DocumentLog (
    ->     log_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     document_id INT,
    ->     old_status VARCHAR(20),
    ->     new_status VARCHAR(20),
    ->     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Trigger

```
mysql> DELIMITER //
mysql> CREATE TRIGGER AfterDocumentUpdate
    -> AFTER UPDATE ON UserDocument
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.status != NEW.status THEN
    ->         INSERT INTO DocumentLog(document_id, old_status, new_status)
    ->         VALUES (OLD.document_id, OLD.status, NEW.status);
    ->     END IF;
    -> END //
Query OK, 0 rows affected (0.03 sec)

mysql> DELIMITER ;
```